RAPPORT DE PROJET

Systèmes d'exploitation (OS)

Application Client-Serveur de chat en Python avec Sockets, Threads et tkinter

Réalisé par : **Seddiki Mohamed**

3ème année Informatique – Option GL (Génie Logiciel)

Code source : https://github.com/Mohamed6seddiki/python-chat-app

Sommaire

1. Introduction 1

2. Notions utilisées 1

3. Architecture 1

4. Le serveur (server.py) 2

5. Le client (client.py) 2

6. Gestion de la concurrence 3

7. Tests et résultats 3

8. Conclusion 3

1. Introduction

Ce projet consiste à réaliser une application de discussion en mode Client-Serveur en Python. Elle est composée de deux programmes indépendants, `server.py` et `client.py`, chacun avec une interface graphique tkinter. Il met en pratique deux notions du cours de systèmes d'exploitation : la communication par sockets TCP et la programmation concurrente avec les threads. Le serveur peut gérer plusieurs clients en même temps sans bloquer son interface.

2. Notions utilisées

- Sockets TCP : point de communication géré par le système d'exploitation, qui garantit une livraison fiable et ordonnée. TCP transmet un flux d'octets et non des messages séparés.
- Appels bloquants : `accept()` et `recv()` suspendent le thread appelant jusqu'à l'arrivée d'une connexion ou de données. Dans le thread de l'interface, ils figeraient la fenêtre, d'où l'utilisation de threads.
- Synchronisation : les threads partagent la mémoire, il faut donc éviter les conditions de concurrence avec un verrou (`threading.Lock`) et une file de messages (`queue.Queue`).

3. Architecture

Dans chaque script, la logique réseau est séparée de l'interface graphique :

Couche	Serveur	Client	Rôle
Logique réseau	`ChatServer`	`ChatClient`	Sockets et threads, sans tkinter

Interface	`ServerApp`	`ClientApp`	Fenêtre tkinter uniquement

tkinter n'est pas thread-safe : les threads de travail ne modifient jamais les widgets. Ils déposent des événements dans une `queue.Queue`, que l'interface vide toutes les 100 ms avec `root.after()`.

Comme TCP ne conserve pas les limites des messages, nous avons défini un protocole simple : chaque message est un texte UTF-8 terminé par `\n`. Le récepteur accumule les octets dans un tampon et le découpe à chaque `\n` :

```
buffer += data while
b"\n" in buffer:
    line, buffer = buffer.split(b"\n", 1)
```

4. Le serveur (server.py)

La fenêtre contient un champ Port, un bouton Listen (puis Stop), un compteur de clients et un journal `ScrolledText` en lecture seule.

- Démarrage : création du socket TCP, `bind` sur `0.0.0.0`, `listen`, puis lancement d'un thread d'acceptation. `SO_REUSEADDR` permet de relancer le serveur sur le même port.
- Acceptation : `accept()` bloque jusqu'à l'arrivée d'un client ; un nouveau thread dédié est alors créé pour lui, et le client est ajouté à un dictionnaire protégé par un `Lock`.
- Service d'un client : le thread lit avec `recv()`, reconstruit les messages grâce au tampon, les écrit dans le journal puis les retransmet aux autres clients avec `sendall()`. Une lecture vide signifie que le client a fermé la connexion.
- Arrêt : on met `running` à `False`, on ferme le socket d'écoute puis chaque connexion avec `shutdown()` et `close()`, ce qui réveille les threads bloqués dans `recv()`.

5. Le client (client.py)

La fenêtre contient les champs Server IP et Port, un bouton Connect (puis Disconnect), l'historique de la discussion, un champ de message et un bouton Send. Le champ de message est désactivé tant que le client n'est pas connecté.

- Connexion : elle se fait dans un thread séparé avec un délai maximal de 5 secondes, pour que la fenêtre reste réactive si le serveur est injoignable.
- Réception : le même thread lit ensuite les données en boucle et envoie chaque message à l'interface par la file d'événements. Si le serveur se ferme, un événement `disconnected` est envoyé.
- Envoi : `sendall()` avec un `\n` final, puis affichage local du message.

6. Gestion de la concurrence

Problème	Solution
Appels bloquants qui figent l'interface	Un thread par tâche bloquante
Dictionnaire des clients partagé	`threading.Lock`
Mise à jour de tkinter depuis un autre thread	`queue.Queue` et `root.after()`
Threads bloqués à l'arrêt	`shutdown()` et `close()`, threads `daemon`
Messages TCP fusionnés ou coupés	Délimiteur `\n` et tampon de réception

7. Tests et résultats

Nous avons testé la partie réseau avec un serveur et deux clients (A et B) : A envoie « hello from A », puis B se déconnecte et le serveur s'arrête. Résultats observés :

- le serveur enregistre les deux connexions (compteur de 2 clients) ;
- le message de A apparaît dans le journal du serveur avec l'adresse de l'expéditeur ;
- le message est retransmis à B uniquement ;
- à l'arrêt du serveur, A reçoit l'événement de déconnexion.

Nous avons aussi vérifié dans l'interface les cas d'erreur : port invalide, port déjà utilisé et connexion à un serveur arrêté. Dans les trois cas, un message d'erreur s'affiche sans plantage ni gel de la fenêtre.

8. Conclusion

Ce projet nous a permis d'appliquer les sockets TCP, les threads, l'exclusion mutuelle et la communication entre threads. L'application gère plusieurs clients simultanément, garde des interfaces réactives et s'arrête proprement. Comme améliorations possibles, on peut ajouter des noms d'utilisateur, le chiffrement avec `ssl`, ou remplacer le modèle « un thread par client » par un modèle asynchrone (`asyncio`) pour supporter plus de clients.

Pour exécuter l'application : lancer `python server.py` et cliquer sur Listen, puis lancer `python client.py`, saisir `127.0.0.1` et le port, et cliquer sur Connect.

Code source du projet : https://github.com/Mohamed6seddiki/python-chat-app